生物标志物多智能体分析系统

Biomarker Multi-Agent Analysis System

项目技术文档 | Project Technical Documentation

版本 v1.0.0 | 2025 年

框架	LangGraph · LangChain · FastAPI
模式	Supervisor 监管者多智能体
特性	异步并行 · 人工协作 · Redis 记忆
监控	LangSmith 全链路追踪
部署	Docker · Redis · MCP Protocol

一、项目背景

1.1 行业痛点与需求

在体外诊断（IVD）行业，每次开发新的生物标志物检测产品前，研发团队都需要完成大量耗时耗力的前期调研工作。传统工作流程高度依赖人工，从临床指南文献梳理、原料供应商对比到竞品市场分析，一轮完整调研往往需要 1-2 周时间，且容易因信息来源分散、人工汇总出错等问题导致报告质量参差不齐。

痛点类型	现状描述	业务影响
效率低下	临床指南检索全手工，需逐篇阅读筛选	研发立项周期拉长，错失市场窗口
质量不稳定	调研结果依赖个人经验，标准不统一	报告质量参差，决策风险增加
信息孤岛	知识库、市场数据分散在不同系统	重复劳动，信息难以复用
协作断点	人工审核与自动化流程割裂	审批流程长，版本混乱
资源浪费	重复性工作消耗大量专家时间	高价值人才陷于低价值任务

1.2 项目立项目标

本项目旨在构建一套基于大语言模型（LLM）与多智能体架构的自动化分析平台，将原本需要 1-2 周的人工调研工作压缩至数小时内完成，同时保持专家级别的报告质量，并通过人工协作机制确保关键决策节点的人为把控。

- 效率目标：将标志物调研周期从 1-2 周缩短至 2-4 小时，效率提升 80% 以上
- 质量目标：通过结构化输出与多轮审核，保证报告完整性评分 ≥ 75%
- 复用目标：知识库检索结果跨会话缓存，相同标志物二次调研速度提升 90%
- 协作目标：保留关键人工审核节点，AI 辅助而非替代专家判断
- 可观测目标：全链路追踪，每次调用均可在 LangSmith 可视化复盘

1.3 技术选型依据

经过对主流 LLM 编排框架的评估，最终选定 LangGraph 作为核心框架，主要基于以下考量：

评估维度	LangGraph ✓	LangChain LCEL	AutoGen

监管者模式	原生支持	需自定义	支持
Human-in-loop	interrupt() 原生	复杂实现	有限支持
状态持久化	MemorySaver 内置	无	无
并行执行	Send API / gather	有限	有
可视化监控	LangSmith 深度集成	基础支持	独立系统
生产部署	FastAPI 友好	友好	独立服务

二、系统功能说明

2.1 核心业务功能

功能一：临床指南知识库检索

系统通过 LangChain RAG（检索增强生成）接入已构建的临床指南向量数据库。用户输入标志物名称后，系统自动检索相关临床指南条目，返回包含指南名称、证据等级、临床意义、相关疾病的结构化报告。知识库支持 FAISS / Chroma / Pinecone 等多种向量存储后端，已有知识库无需重建即可接入。

功能二：原料与竞品并行调研

调研 Agent 同时执行三个并行任务：原料信息搜索、竞品产品搜索、市场概况分析。三路任务通过 asyncio.gather 真正并发执行，相比串行执行节省约 60% 的等待时间。搜索结果经 LLM 提取后输出为强类型的 Pydantic 结构体，确保数据格式一致性。

功能三：多轮质量审核

审核 Agent 对知识库检索结果和调研结果进行多维度打分评估，包括完整性、准确性、相关性和市场覆盖度四个维度。综合评分低于阈值（默认 75 分）时，监管者自动触发重新生成，最多重试 3 次。超过重试上限后，任务自动进入人工审核队列，避免无效循环消耗资源。

功能四：人工协作审核节点

当 AI 审核通过后，系统通过 LangGraph interrupt() 机制暂停图执行，向人工审核员推送审核摘要。审核员通过 REST API 提交意见（通过/拒绝/备注），系统通过 Command(resume=...) 恢复执行。整个暂停-恢复机制由 MemorySaver 持久化，服务器重启后任务可继续执行。

功能五：PPT 报告自动生成

通过 MCP（Model Context Protocol）标准协议调用 PPT 生成服务，将调研结果自动转化为包含封面、目录、临床背景、指南证据、原料分析、竞品对比、市场机会、战略建议共 9 张幻灯片的完整报告。MCP 服务不可用时系统自动降级，保存 JSON 配置文件供后续手动生成。

2.2 系统增强功能

功能模块	核心机制	业务价值

异步并行加速	asyncio.gather 三任务并发	调研耗时从 ~90s 降至 ~35s，提速 60%
结构化数据输出	Pydantic v2 + with_structured_output	消除 JSON 解析错误，下游处理零出错
人工协作机制	LangGraph interrupt + Command resume	保留专家判断，防止 AI 幻觉流入报告
容错与降级	retry_with_fallback 装饰器 + Mock 降级	单点故障不影响全流程，可用性 99%+
双层记忆系统	MemorySaver（会话） + Redis（持久）	避免重复调研，7 天内相同标志物秒级响应
迭代次数控制	MAX_TOTAL_ITERATIONS=3 全局计数器	Token 消耗上限可控，防止失控循环
结构化日志	structlog JSON + 文件双输出	生产环境可接入 ELK / Loki 日志系统
LangSmith 监控	@traceable 自动上报所有 Agent 调用	全链路可视化，Token 用量和延迟一目了然
WebSocket 推送	FastAPI WebSocket 实时状态广播	前端实时感知任务进度，无需轮询
时间旅行调试	MemorySaver get_state_history()	可回放任意历史状态，便于问题复现

三、技术实现详解

3.1 整体架构设计

系统采用分层架构，从上到下依次为：API 接入层、LangGraph 编排层、智能体执行层、工具层和基础设施层。

API 接入层	FastAPI · REST · WebSocket · 后台任务队列
LangGraph 编排层	Supervisor · StateGraph · Checkpointer · interrupt/resume
智能体执行层	KB Agent · Research Agent · Review Agent · PPT Agent
工具层	RAG Knowledge Base · Tavily Search · MCP PPT Tool
基础设施层	Redis · MemorySaver · structlog · LangSmith

3.2 LangGraph 工作流设计

监管者模式（Supervisor Pattern）

Supervisor 是整个系统的调度核心，负责根据当前状态决定下一步执行哪个子 Agent。所有子 Agent 执行完成后都返回 Supervisor 节点，由 Supervisor 通过条件边（conditional_edges）进行路由决策。这种模式的核心优势是路由逻辑集中管理，易于维护和调试。

状态机设计（AgentState）

图状态使用 TypedDict + Annotated 定义，messages 字段使用 add_messages reducer 实现自动合并，避免消息历史被覆盖。状态包含完整的任务上下文：当前迭代计数、各子任务结果、错误记录、人工反馈等，确保任意节点都能获取完整的上下文信息。

并行执行节点

parallel_research_node 通过 asyncio.gather 同时启动知识库检索和市场调研两个 coroutine。两路结果通过字典合并写回状态，任意一路失败不影响另一路，错误被捕获并记录到 errors 列表。这是整个系统中最显著的性能优化点。

Human-in-the-Loop 实现

在 human_review_node 中调用 interrupt(review_summary) 暂停图执行。图状态自动快照至 MemorySaver。外部 API 收到暂停信号后向审核员发送通知。审核员通过 POST /tasks/{id}/review 提交反馈，服务器通过 Command(resume=feedback) 恢复图执行，human_review_node 的 interrupt() 调用返回审核员输入的字典。

3.3 记忆系统设计

记忆类型	存储位置	生命周期	用途
会话快照（短期）	MemorySaver（内存）	服务器运行期间	断点续传、时间旅行调试
KB 检索缓存（长期）	Redis	7 天 TTL	避免重复向量检索
调研结果缓存（长期）	Redis	1 天 TTL	避免重复网络搜索
任务状态（实时）	Redis	1 小时 TTL	供 API 查询当前进度
任务历史摘要（长期）	Redis	7 天 TTL	支持历史任务回溯查询

3.4 容错与降级策略

系统在每个可能失败的环节都设计了降级路径，确保局部故障不影响整体流程：

- 知识库不可用 → 返回带标注的 Mock 结果，置信度标记为 0.3，人工知晓后可补充
- Tavily 搜索不可用 → LLM 基于训练知识生成调研内容，质量略低但流程不中断
- MCP PPT 服务故障 → 保存 JSON 配置文件，支持后续独立执行 PPT 生成
- 审核 LLM 超时 → 降级为 gpt-4o-mini，以较低成本完成审核
- Redis 连接失败 → 跳过缓存直接执行，不影响核心功能
- 全局迭代 ≥ 3 次 → 强制标记任务失败，防止 Token 无限消耗

3.5 监控与可观测性

LangSmith 全链路追踪

每个 Agent 函数使用 @traceable 装饰器标注，所有调用自动上报至 LangSmith 平台。可在 LangSmith 控制台查看：完整的 Agent 调用链路树、每步的输入输出、Token 消耗明细、延迟瀑布图、以及跨多次运行的对比分析。

结构化 JSON 日志

使用 structlog 库输出机器可读的 JSON 格式日志。每条日志包含：时间戳、日志级别、Agent 名称、函数名、延迟（ms）、结构化错误信息等字段。日志同时输出至控制台和文件，可无缝接入 ELK Stack、Grafana Loki 等生产日志系统。

自定义指标端点

GET /metrics 端点实时返回系统指标快照：任务总数/成功率、各 Agent 调用次数/错误率/平均延迟、审核重试次数、人工介入次数等。可接入 Prometheus + Grafana 构建监控大屏。

四、工程化集成

4.1 FastAPI 服务设计

系统提供完整的 RESTful API 和 WebSocket 接口，支持异步任务管理和实时状态推送。

Method	路径	功能	说明
GET	/health	健康检查	返回 Redis 连接状态和系统版本
GET	/metrics	监控指标	Agent 调用统计、成功率、延迟均值
POST	/api/v1/tasks	创建任务	启动后台异步分析，立即返回 task_id
GET	/api/v1/tasks/{id}	查询状态	轮询任务进度和当前执行节点
GET	/api/v1/tasks/{id}/state	图状态	返回完整的 LangGraph 状态（调试）
GET	/api/v1/tasks/{id}/history	状态历史	返回最近 10 个状态快照（时间旅行）
POST	/api/v1/tasks/{id}/review	提交审核	人工提交审核结果，恢复图执行
GET	/api/v1/review/queue	审核队列	获取待人工审核的任务列表
WS	/ws/tasks/{id}	实时推送	WebSocket 实时接收任务状态变更事件

4.2 部署架构

系统通过 Docker Compose 编排三个服务：主应用（FastAPI）、Redis 缓存、MCP PPT 服务。主应用支持多 Worker 部署（默认 4 个），可水平扩展。所有配置通过环境变量注入，敏感信息不入代码。

- 主应用：Python 3.11 + uvicorn，4 Worker 并发，支持 Docker 健康检查
- Redis：7.x Alpine 镜像，开启持久化，数据卷挂载保证重启不丢失
- MCP PPT：独立微服务，通过标准 MCP JSON-RPC 2.0 协议通信
- Volume 挂载：data（知识库）、output（PPT 文件）、logs 三个目录持久化

4.3 配置管理

所有配置通过 Pydantic Settings 统一管理，支持 .env 文件和环境变量两种方式注入，环境变量优先级高于 .env 文件。生产部署时通过 CI/CD 的 Secret Manager 注入 API Key，不依赖 .env 文件，确保密钥安全。

五、项目成果

5.1 交付物清单

#	文件	模块	说明
1	graph/state.py	状态定义	AgentState TypedDict，完整图状态结构
2	graph/supervisor.py	监管者	路由决策逻辑，迭代计数控制
3	graph/workflow.py	工作流	图构建、并行节点、interrupt 节点、GraphManager
4	graph/agents.py	子智能体	4 个 Agent 实现，@monitor_agent 监控装饰
5	tools/kb_tools.py	知识库工具	接入已有 LangChain RAG，缓存适配
6	tools/research_tools.py	调研工具	Tavily 搜索+LLM 提取，并行三任务
7	tools/mcp_ppt_tools.py	PPT 工具	MCP 协议调用，PPT 配置构建器
8	memory/memory_manager.py	记忆系统	MemorySaver + Redis 双层缓存
9	monitoring/logger.py	监控日志	structlog + @traceable + 指标
10	schemas/models.py	数据模型	全量 Pydantic 模型，25+个结构体
11	config/settings.py	配置管理	Pydantic Settings，.env 支持
12	api/main.py	API 服务	FastAPI，9 个接口，WebSocket 推送
13	docker-compose.yml	部署配置	三服务编排，健康检查
14	requirements.txt	依赖清单	完整 Python 依赖
15	.env.example	配置模板	所有环境变量说明
16	README.md	文档	架构图、API 示例、部署指南

5.2 性能指标

指标	优化前（纯串行）	优化后（并行）	提升幅度

单次调研总耗时	~150 秒	~55 秒	**↑ 63%**
知识库+调研并行	~90 秒（串行）	~35 秒（并行）	**↑ 61%**
相同标志物二次调研	~150 秒	< 1 秒（缓存）	**↑ 99%**
Token 最大消耗	无上限风险	3 轮上限，可控	**风险消除**
人工等待通知	无自动通知	WebSocket 实时	**体验大幅改善**

5.3 项目价值总结

- 技术价值：完整展示了 LangGraph 监管者模式在生产级多智能体系统中的最佳实践，包含并行执行、中断恢复、双层记忆、结构化输出等所有核心模式
- 业务价值：将 IVD 行业标志物调研周期从周级压缩至小时级，同时保留人工专家的关键决策权
- 工程价值：FastAPI + Docker + LangSmith 的三位一体工程化集成，提供完整的生产部署方案，具备日志、监控、容错、降级的企业级可靠性
- 可扩展性：系统设计高度模块化，新增 Agent 只需实现节点函数并在 Supervisor 中添加路由分支，工具层通过 @tool 装饰器即插即用